

UniVerse

A Campus Companion

The Search & Campus-Explore System

Find Teacher * Room Availability

Leading University, Sylhet - Dept. of CSE

Course CSE-3240 (Project I) - Team Sherlocked

How search works - the code files & functions behind it -
the full frontend-to-backend connection - and how data is
retrieved in real time, with visual diagrams.

1. What 'Search' Means in UniVerse

UniVerse's search system is the **Campus Explore** feature - reached from the floating **Explore** button on every dashboard. It has two faces that share one engine:

- **Find Teacher** - type a name or teacher code and instantly see *where that teacher is right now* (which room, time remaining) or *when their next class is*.
- **Room Availability** - browse every room and see whether it is **OCCUPIED** or **AVAILABLE** right now, with the current class and the next one.

The clever part: there is **no separate 'search index' or extra database table**. Both features read only the existing **routines** table (the published weekly timetable) plus **cancellations**, and *derive* live status by comparing the current clock time against the schedule. Search is therefore two things working together:

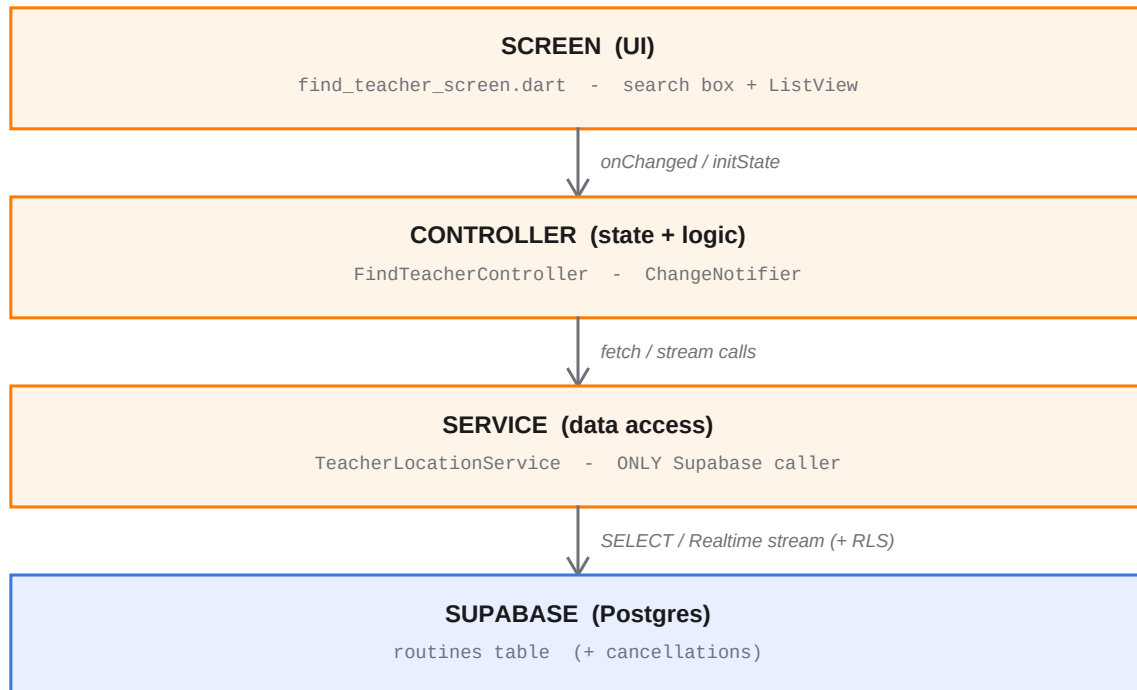
(a) Retrieval - pull the weekly schedule from Supabase (once, then live updates).

(b) Filtering + derivation - in the app, narrow the list by the typed query and compute each teacher's / room's current state from ``DateTime.now()``.

Key idea: Search = retrieve the timetable once, keep it live, then filter & time-derive on the device. No search server, no extra tables.

2. The Layered Architecture (Screen -> Controller -> Service -> DB)

UniVerse enforces a strict one-way layering. The screen never talks to the database; it goes through a controller, which goes through a service, which is the **only** layer allowed to touch Supabase.



Why this matters for retrieval: because only the service touches Supabase, every query automatically runs under the user's session - so **Row Level Security** decides what rows come back. The routines table is ``read_all`` for any authenticated user, which is exactly why a student can look up **any** teacher or room.

3. The Code Files & Functions Behind It

Find Teacher and Room Availability are mirror-image feature slices. Here is the full map:

Find Teacher ([lib/features/find_teacher/](#))

File	Responsibility / key functions
<code>screens/find_teacher_screen.dart</code>	Search box (UITextField) + live ListView of teacher cards. initState() starts the controller + real-time subscription; onChanged calls controller.search().
<code>controllers/find_teacher_controller.dart</code>	Brain. initialize() builds the teacher list; _buildTeacherInfo() derives In Class / Free / No Class Today from the clock; search() filters by name or code; subscribeToRealTimeUpdates() keeps it live. Holds TeacherInfo model.
<code>services/teacher_location_service.dart</code>	ONLY Supabase layer. fetchAllRoutines() (one-shot SELECT) and streamAllRoutines() (Realtime). Maps rows -> RoutineEntry.
<code>widgets/teacher_location_card.dart</code>	Presentational card: room, remaining time, next class, status chip.

Room Availability ([lib/features/rooms/](#))

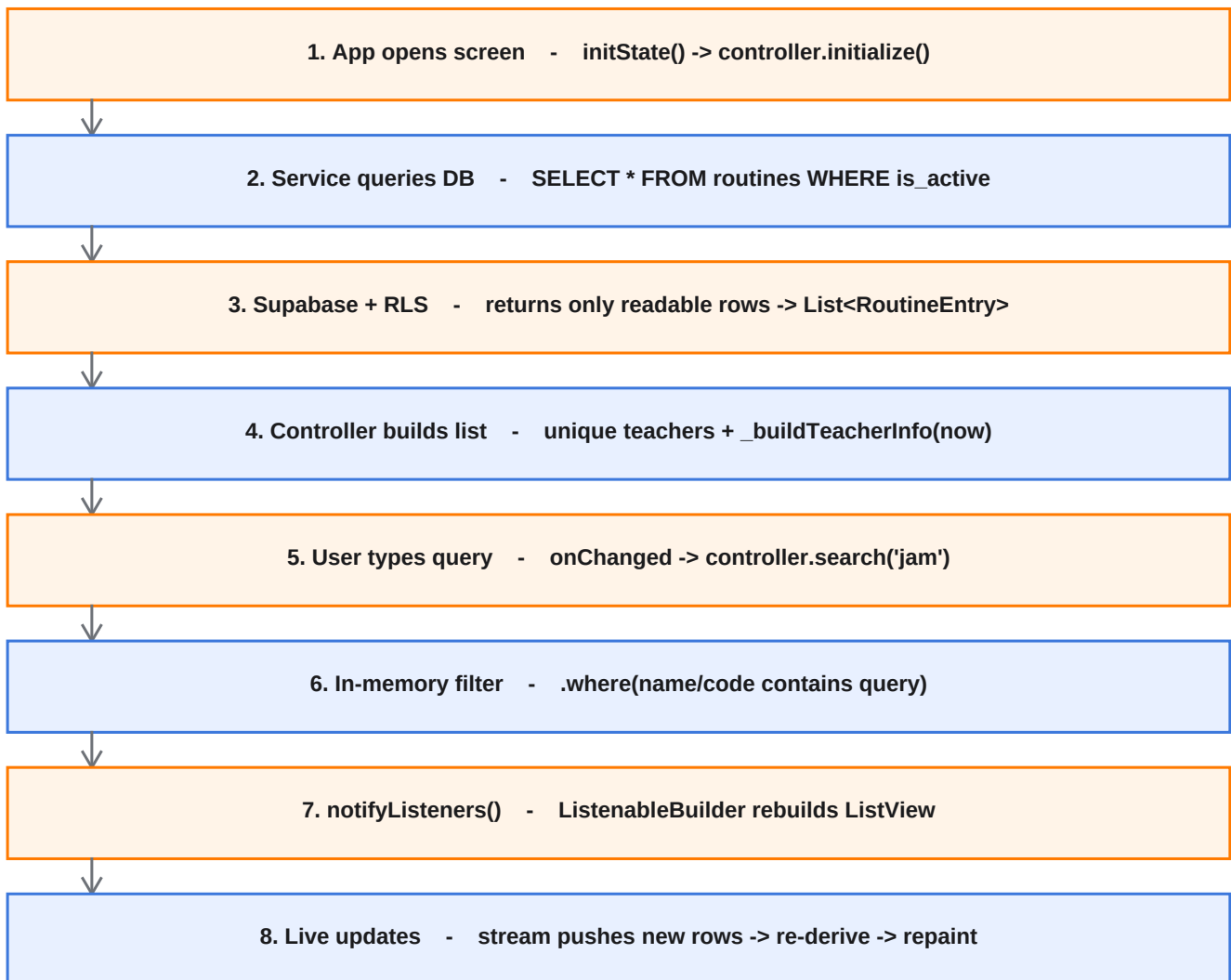
File	Responsibility / key functions
<code>screens/rooms_screen.dart</code>	Room list with OCCUPIED / AVAILABLE chips.
<code>controllers/rooms_controller.dart</code>	fetchRoomStatuses() groups routines by room, then time-derives each room's current + next class. Holds RoomStatus model.
<code>widgets/room_status_card.dart</code>	ONLY Supabase layer. fetchAllRoutines() + streamAllRoutines() on routines. Occupancy chip, current teacher/subject, countdown.

Shared foundations

File	Responsibility / key functions
<code>core/models/routine_model.dart</code>	RoutineEntry.fromMap() parses a DB row; startOn(now)/endOn(now) turn time strings into today's DateTime for comparison.
<code>shared/widgets/explore_fab_menu.dart</code>	The Explore FAB -> bottom sheet -> pushes /find-teacher or /rooms.

4. Full Frontend <-> Backend Flow (Find Teacher)

This is the complete journey of one search, from a keystroke to the rendered card.



Steps 1-4 are **retrieval** (frontend asking the backend). Steps 5-7 are **search** (pure on-device filtering - no network). Step 8 is **real-time retrieval** keeping the result fresh.

5. How Data Is Retrieved (the backend call)

5.1 One-shot fetch

The service issues a single filtered SELECT. `.eq('is_active', true)` skips superseded timetables; ordering by start time makes the later 'current/next' scan simple.

```
// teacher_location_service.dart
Future<List<RoutineEntry>> fetchAllRoutines() async {
  final rows = await _supabase
    .from(AppConstants.tableRoutines)
    .select()
    .eq('is_active', true)
    .order('time_start', ascending: true);
  return _map(rows); // each row -> RoutineEntry.fromMap()
}
```

5.2 Real-time retrieval (the 'live' in live search)

Instead of polling, the service opens a Supabase **Realtime stream**. Whenever the routines table changes (e.g. admin publishes a new timetable), Supabase pushes the new row set; the controller re-derives every status and the UI repaints - with no user action.

```
// teacher_location_service.dart
Stream<List<RoutineEntry>> streamAllRoutines() {
  return _supabase
    .from(AppConstants.tableRoutines)
    .stream(primaryKey: ['id'])
    .eq('is_active', true)
    .map((rows) => _map(rows));
}

// controller subscribes once:
void subscribeToRealTimeUpdates() {
  _service.streamAllRoutines().listen((routines) {
    _allRoutines = routines;
    initialize(); // rebuild + re-derive
  });
}
```

Key idea: Realtime streams obey RLS too - the live feed only carries rows the user is allowed to read.

6. The Search Filter & Time-Derivation Logic

6.1 The actual search (substring match, case-insensitive)

Search is a pure in-memory filter - instant, offline, no DB round-trip. It matches the lowercased query against either the teacher's name or code:

```
// find_teacher_controller.dart
void search(String query) {
  _searchQuery = query.toLowerCase();
  if (_searchQuery.isEmpty) {
    _filteredTeachers = _allTeachers;
  } else {
    _filteredTeachers = _allTeachers.where((t) =>
      t.name.toLowerCase().contains(_searchQuery) ||
      t.code.toLowerCase().contains(_searchQuery)).toList();
  }
  notifyListeners(); // -> UI rebuilds the filtered list
}
```

6.2 Deriving 'where is this teacher right now?'

For each teacher, the controller keeps only today's classes, sorts them, then walks them against `DateTime.now()`:

- If now is **between** a class start and end -> status **In Class** (show room + minutes remaining).
- Else the first class still in the future -> status **Free** (show next class).
- No classes today -> **No Class Today**.

```
// _buildTeacherInfo(...) - the core decision
for (final entry in teacherRoutines) {
  final start = entry.startOn(now);
  final end   = entry.endOn(now);
  if (now.isAfter(start) && now.isBefore(end)) {
    currentEntry = entry; status = 'In Class'; break;
  }
}
```

'Today' is matched by mapping `DateTime.now().weekday` onto the Sun-Sat day names that the routines table stores - the project's single 7-day source of truth.

7. Room Availability - Same Engine, Grouped by Room

Room Availability reuses the identical retrieval path, but the controller **groups routines by room** instead of by teacher, then time-derives occupancy:

```
// rooms_controller.dart (shape)
routinesByRoom[entry.room].add(entry);    // group
for (room in routinesByRoom) {
  if (now between start & end) -> OCCUPIED (current class)
  else -> AVAILABLE (+ next class if any)
}
```

So the two features differ only in the *grouping key* (teacher_code vs room) and what each card shows. Both are read-only, real-time, and add no new tables - exactly as designed.

Key idea: One retrieval engine, two lenses: group by teacher = Find Teacher; group by room = Room Availability.

8. One-Page Summary (for the viva)

What is the search system? The Campus Explore feature: Find Teacher (search by name/code) and Room Availability, both real-time.

Where does the data come from? Only the existing `routines` table (+ `cancellations`) via a service layer - no search engine, no extra tables.

Frontend-to-backend path: Screen -> Controller -> Service -> Supabase (under RLS) -> back as RoutineEntry objects.

How is it retrieved? A filtered SELECT for the first paint, then a Supabase Realtime stream for live updates.

How does search itself work? A pure in-memory, case-insensitive substring filter on the already-loaded list (instant, offline).

How is 'right now' known? The controller compares `DateTime.now()` to each class's start/end to derive In Class / Free / Occupied / Available.

Why this design? It is simple, needs no scheduled jobs, stays correct the instant the admin publishes a routine, and respects per-role security automatically through RLS.

Files to point at: find_teacher_screen.dart (UI+search box), find_teacher_controller.dart (search() + _buildTeacherInfo()), teacher_location_service.dart (fetchAllRoutines / streamAllRoutines), and the rooms/ mirror.